eMerger

A one-command system updater

Roberto Antoniello & Edoardo Ferrari

October 29, 2026



Abstract

eMerger is a command line tool for Linux, macOS and Windows that refreshes, upgrades and cleans a system with one command, `up`. It detects the package managers installed on the host, runs them in order and prints one summary at the end. Bash on Unix, PowerShell on Windows.

This manual covers every platform: every flag, configuration variable, environment variable, file, exit code and JSON field, and the plugin API.

Contents

1 Overview **2**
- 1.1 Background . . . 2
- 1.2 What up does . . . 2

2 Platforms **2**

3 Requirements **3**
- 3.1 Linux . . . 3
- 3.2 macOS . . . 3
- 3.3 Windows . . . 3

4 Installation **4**

4.1 Linux and macOS . . . 4
4.2 Windows . . . 4
4.3 Manual install . . . 5
4.4 Uninstall . . . 5

5 Self-update **5**

6 Quickstart **5**
6.1 Step 1: dry-run . . . 5
6.2 Step 2: real run . . . 5
6.3 Step 3: see what changed . . . 5
6.4 Step 4 (optional): weekly unattended update . . . 6

7 Example run **6**

8 Execution flow **6**

9 Command line reference **7**
9.1 All flags . . . 7
9.2 Combining flags . . . 8

10 Features **9**
10.1 Dry-run and verbose . . . 9
10.2 Quiet levels . . . 9
10.3 Security-only updates . . . 9
10.4 Dev toolchains . . . 9
10.5 Firmware (Linux) . . . 10
10.6 Raspberry Pi . . . 10
10.7 Live progress . . . 10
10.8 Cache cleaning . . . 11
10.9 Deep clean . . . 11
10.10 Parallel mode . . . 12
10.11 Snapshots and rollback (Linux) . . . 12
10.12 Mirrors refresh (Linux) . . . 12
10.13 Reboot handling . . . 12
10.14 Resume . . . 13
10.15 Package diff and changelog . . . 13
10.16 History and errors . . . 13
10.17 Doctor . . . 13

11 Configuration **13**
11.1 Config file (Unix) . . . 13
11.2 Config file (Windows) . . . 14
11.3 Every supported variable . . . 14
11.4 Profiles . . . 16

12 Hooks **17**
12.1 Example: back up dotfiles before every run . . . 17
12.2 Example: Slack notification after every run . . . 17
12.3 Example: copy the last run log to the clipboard (Windows) . . . 18
12.4 Ignore list (Linux) . . . 18
12.5 Quiet hours . . . 18

13 Manager plugins **18**

13.1 Minimal plugin . . . 18
13.2 Plugin API . . . 19
13.3 Behavior . . . 19
13.4 Detection cache . . . 19

14 Integration **19**
14.1 JSON output . . . 19
14.2 Prometheus metrics . . . 20
14.3 Reboot exit code . . . 20
14.4 Download-only . . . 20
14.5 Manager filtering . . . 21

15 Auto-update (unattended) **21**
15.1 Managing the schedule . . . 21

16 Cookbook **22**
16.1 Interactive run . . . 22
16.2 Unattended server . . . 22
16.3 Dev toolchains and firmware . . . 22
16.4 Least change . . . 22
16.5 Download now, install later . . . 22
16.6 CI runner and container image build . . . 22
16.7 Snapshot and rollback . . . 22
16.8 One package . . . 23

17 Security model **23**

18 Files and paths **24**

19 Supported package managers **24**

20 Exit codes **25**

21 Troubleshooting **25**

22 Frequently asked questions **26**

23 Development **27**
23.1 Repository layout . . . 27
23.2 Bash library modules (`src/lib/`) . . . 27
23.3 PowerShell library modules (`src/pslib/`) . . . 29
23.4 Running tests . . . 29
23.5 Adding a package manager . . . 29
23.6 Contributing . . . 30

1 Overview

1.1 Background

Your software comes from lots of places:

- the operating system (`apt`, `dnf`, `pacman`, `zypper`, `softwareupdate`, `winget` and others);
- cross-distribution app stores (`flatpak`, `snap`, `brew --cask`, `choco`, `scoop`);
- user-level stores (`brew`, `mas`);
- language ecosystems (`npm`, `pip`, `cargo`, `gem`, `pnpm` and others);
- vendor firmware (`fwupdmgr` on Linux, `softwareupdate` on macOS, Dell/Lenovo utilities on Windows).

Each has its own commands, its own cache and its own idea of what a security update or a clean-up is. eMerger puts them behind one command.

1.2 What `up` does

On any supported platform, `up`:

1. detects the package managers installed on the host;
2. asks for `sudo` (or a UAC elevation on Windows) only if a detected manager needs it;
3. optionally takes a filesystem snapshot, and records the package list so the run can be audited afterwards;
4. runs refresh, upgrade and clean for each manager, serially or in parallel where safe;
5. optionally cleans user caches and the trash;
6. prints a summary with the result per manager, disk freed, run duration and whether a reboot is needed;
7. on request, exports the same summary as JSON or a Prometheus textfile.

2 Platforms

	Linux	**macOS**	**Windows**
Entry point	`src/emerger.sh`	`src/emerger.sh`	`src/emerger.ps1`
Shell	bash 4.4+	bash 4.4+ (Homebrew)	PowerShell 5.1+
Install	`./install.sh`	`./install.sh`	`.\install.ps1`
Uninstall	`./uninstall.sh`	`./uninstall.sh`	`.\uninstall.ps1`
Auto-update	systemd timer / cron	cron	Task Scheduler
Elevation	`sudo`	`sudo`	UAC, admin managers only
Parallel mode	yes	yes	no (always serial)
Snapshots	snapper/timeshift/btrfs	no	no
Config dir	`~/.config/emerger/`	`~/.config/emerger/`	`%APPDATA%\emerger\`
State dir	`~/.local/state/emerger/`	`~/.local/state/emerger/`	`%LOCALAPPDATA%\emerger\state\`

The core flow (detect, upgrade, clean, summary) is the same everywhere. Platform-specific features are marked as such.

3 Requirements

eMerger does not install any of these; it uses whatever is there.

3.1 Linux

Required: `bash` 4.4+, `coreutils`, `git`, `sudo`.

Optional:

- `notify-send` for desktop notifications.
- `curl` for the weather line (`-w`) and the `--doctor` connectivity check.
- `flock` for the global lock (part of `util-linux`, installed by default on most distributions). Without it the lock is the directory `emerger.lock.d`.
- `timeout` (coreutils) for `MANAGER_TIMEOUT`.
- `pkill` (procps) to stop the `--parallel` managers when a run is interrupted; without it their commands finish first.
- `reflector` (Arch) or `netselect-apt` (Debian) for `--refresh-mirrors`.
- `snapper`, `timeshift` or `btrfs-progs` for `--snapshot` and `--rollback`.
- `fwupdmgr` for `--firmware`.

3.2 macOS

Required: `bash` 4.4 or later, and Xcode Command Line Tools for `git`. The system `/bin/bash` is 3.2 and is too old (eMerger uses `mapfile`, `${var,,}`, `{fd}` redirections and empty-array expansion under `set -u`); install a current one with `brew install bash` and make sure it comes first in `PATH`, since the `up` alias runs `bash` from `PATH`.

Optional:

- Homebrew (`brew`), which also enables `brew --cask`.
- `mas` (`brew install mas`) for Mac App Store upgrades.

3.3 Windows

Required: PowerShell 5.1 (built-in on Windows 10 and later) or 7+, Git for Windows (needed by `up --self-update`).

Optional:

- `winget`, `scoop`, `choco`: any combination is auto-detected.
- `PSWindowsUpdate` (`Install-Module PSWindowsUpdate -Scope CurrentUser`) for Windows Update integration.
- `BurntToast` (`Install-Module BurntToast -Scope CurrentUser`) for native toast notifications.

4 Installation

4.1 Linux and macOS

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
./install.sh
# open a new shell, or: source ~/.bashrc
up --help
```

`install.sh`:

1. Adds `alias up='bash "/path/to/eMerger/src/emerger.sh"'` to your shell rc files (`.bashrc`, `.zshrc`, `config.fish`). A checkout path containing a single or double quote, `$`, a backquote, `%`, a backslash or a newline is refused.
2. Marks `src/emerger.sh` as executable.
3. Installs shell completions for `bash`, `zsh` and `fish`:
   - `~/.local/share/bash-completion/completions/up`
   - `~/.zsh/completions/_up`
   - `~/.config/fish/completions/up.fish`
4. Installs the man page as `~/.local/share/man/man1/up.1`, so `man up` shows it; if it does not, add `~/.local/share/man` to `MANPATH`.
5. Creates `~/.config/emerger/` with `config.sh`, `ignore.list`, `hooks/` and `profiles.d/`.

On macOS the alias goes to `~/.zshrc` first, because `zsh` is the default shell since Catalina. If Homebrew is present, completions are installed under the Homebrew prefix.

4.2 Windows

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
.\install.ps1
# open a new PowerShell window, or: . $PROFILE
up --help
```

`install.ps1` does not need administrator rights. It:

1. When `ExecutionPolicy` for `CurrentUser` is `Restricted` or `Undefined`, asks before setting it to `RemoteSigned`; no answer (or no terminal) leaves it unchanged.
2. Adds an `up` function that calls `emerger.ps1` to `$PROFILE.CurrentUserAllHosts`, so `up` works in every PowerShell host (console, ISE, Windows Terminal, the VS Code terminal).
3. Creates `%APPDATA%\emerger\` with `config.ps1` and the `hooks` and `profiles.d` folders.

When admin managers (choco, wsl, Windows Update) are detected, eMerger runs them in one elevated child behind a single UAC prompt. The child loads no `config.ps1`, profile or hook; its results, exit status and log lines come back to the unelevated run, which prints them.

4.3 Manual install

Without the install script, point an alias or function at the entry point:

```
# bash, zsh, fish (Linux or macOS)
alias up='bash /absolute/path/to/eMerger/src/emerger.sh'
```

```
# PowerShell (Windows)
function up { & 'C:\path\to\eMerger\src\emerger.ps1' @args }
```

4.4 Uninstall

```
./uninstall.sh      # Linux / macOS
.\uninstall.ps1     # Windows
```

Removes the alias or up function, the completion files, the man page and any cron job, systemd user timer or scheduled task. It then asks whether to delete the settings, history and cache too; the answer defaults to no, and without a terminal they are kept. To remove them later by hand:

```
rm -rf ~/.config/emerger ~/.local/state/emerger ~/.cache/emerger
```

The repo itself stays where it is; delete it by hand.

5 Self-update

```
up -up                # short flag
up --self-update      # long flag
```

This runs `git pull --ff-only` in the clone and prints the new commit range. Non fast-forward updates are refused, so local changes are never overwritten.

6 Quickstart

Steps 1 and 3 change nothing on the system.

6.1 Step 1: dry-run

```
up -n
```

Shows the detected managers and the commands that would run. It does not ask for sudo, download anything or write to disk.

6.2 Step 2: real run

```
up
```

Refreshes, upgrades and cleans each manager in turn. You get one sudo prompt (Unix) or UAC prompt (Windows) if a manager needs it.

6.3 Step 3: see what changed

```
up --changed
```

Prints the package diff for the last run with + for added, - for removed and ~ for upgraded.

6.4 Step 4 (optional): weekly unattended update

```
up -au
```

Installs a systemd user timer (Linux), cron job (macOS) or scheduled task (Windows): Sundays at 10:00 local time, with `-y -q -nl -ni`. The timer and the task start up to one hour later at random; cron starts at 10:00 sharp.

7 Example run

Output with apt and flatpak:

```
$ up
  (gradient-colored eMerger logo)
  * Debian GNU/Linux 13 . x86_64 . 2026-10-16 10:12 . eMerger v3.0.0

  * apt  ~40s ---------------------------------------- #. 1/2
    [OK] fix broken ..............   1.2s
    [OK] update ..................   6.8s
    .:. full-upgrade  .:::.   0:14  Unpacking libssl3 ...

  * flatpak ------------------------------------------ ## 2/2
    [OK] update ..................  12.4s  5 refreshed

  > User cache ------------------------------------------------
    ? Clean user cache? (812M) [y/N] y
    [OK] cache cleaned ............  812M

  +- eMerger summary --------------------------+
  | managers  [OK] apt  [OK] flatpak           |
  | time      0m42s                            |
  | freed     812.0MB                          |
  | status    [OK] all good                    |
  +--------------------------------------------+
```

This is the ASCII glyph set (`--ascii` or a non-UTF-8 locale). In a UTF-8 terminal you get rounded box corners, check marks, a braille spinner and the `.:::.` wave; per-manager emoji icons need a color-emoji font, otherwise plain symbols are used. A running step shows the elapsed time and the last line the manager printed, then collapses to one result line with duration and a short stat.

> Don't run `sudo up`. eMerger asks for `sudo` itself, only for the managers that need it. Under `sudo up`, user-level packages (`pip --user`, Homebrew, `npm -g`) end up in root's home.

8 Execution flow

In order:

1. Load `config.sh` (or `config.ps1`) if present.
2. If `--profile NAME` is on the command line, pre-load `share/profiles/NAME.sh` (or the user override from `~/.config/emerger/profiles.d/`).
3. Parse CLI flags (they override config and profile).
4. Unix: if `QUIET_HOURS` is set, `-y` is set and the clock is inside the window, exit 0. Scheduled runs are skipped, interactive ones are not.

5. Acquire the global exclusive lock (`~/.local/state/emerger/emerger.lock` via `flock` on Unix, `%LOCALAPPDATA%\emerger\state\run.lock` on Windows). If another `up` is already running, abort with exit code 1.
6. Print the logo, OS info line and the current timestamp.
7. Warn on low battery (<20% on a laptop) and on low free disk space (default: less than 1024 MiB on `/` or `C:`). On a Raspberry Pi, check power and temperature.
8. Cache `sudo` credentials on Unix, only when a detected manager needs it. On Windows the admin managers run later in one elevated child.
9. Unix: with `--refresh-mirrors`, re-rank the mirrors; with `--snapshot`, take a snapshot or exit 1.
10. Snapshot the installed-packages list into `pkgs.before` for the post-run diff.
11. Execute `pre.d` hooks in alphabetical order.
12. For each detected manager, run *refresh*, *upgrade*, *clean*; serial for system managers, concurrent for safe user-space ones when `--parallel` is active.
13. Optionally clear user cache and trash (skippable with `-nc` and `-nt`), then, with `--deep-clean`, the system leftovers and the container data (Unix).
14. Execute `post.d` hooks.
15. Snapshot the installed-packages list into `pkgs.after` and compute `pkgs.diff`.
16. Render the boxed summary, the reboot advisory and an optional desktop notification.
17. Exit 0 on success, 3 if any manager failed, 4 if a reboot is required and `--reboot-exit` was passed.

9 Command line reference

9.1 All flags

Flag	Meaning	Platforms
`-h`, `--help`	Print help text and exit	all
`-V`, `--version`	Print `eMerger X.Y.Z`	all
`-n`, `--dry-run`	Preview without running	all
`-v`, `--verbose`	Stream manager output live	all
`-q`, `--quiet`, `-qq`, `-qqq`	Quieter by one level per `-q`	all
`-y`, `--yes`	Assume yes on prompts	all
`-w`, `--weather`	Show weather line via wttr.in	all
`--ascii`	Force ASCII glyphs only	all
`--deep-clean`	System leftovers (see §10.9)	Linux/macOS
`-nl`, `--no-logo`	Hide logo	all
`-ni`, `--no-info`	Hide system info line	all
`-nc`, `--keep-cache`	Skip user cache cleaning and the warning about space held by deleted files still open	all
`-nt`, `--keep-trash`	Skip trash cleaning	all
`--security`	Security-only updates	supported managers

Flag	Meaning	Platforms
`--firmware`	Include `fwupdmgr`	Linux
`--no-firmware`	Force-skip firmware	Linux
`--dev`	Include dev toolchains	all
`--parallel`	Run user-space in parallel	Unix
`--profile NAME`	Load a profile	all
`--only LIST`	Only these managers	all
`--except LIST`	Skip these managers	all
`--snapshot`	snapper / timeshift / btrfs	Linux
`--refresh-mirrors`	Re-rank distro mirrors	Linux
`--resume`	Skip completed managers	Unix
`--reboot-exit`	Exit 4 if reboot required	all
`--download-only`	Prefetch, don't install	Unix
`--accept-licenses`	Accept licence agreements	macOS, Windows
`-err`, `--errors`	Tail of logged errors	all
`--json`	Machine-readable summary	all
`-up`, `--self-update`	Self-update eMerger	all
`-au`, `--auto-update`	Install weekly auto-update	all
`-rc`, `--rebuild-cache`	Clear detection cache (Windows keeps none)	all
`--`	End of options: the words after it are ignored	all

On Windows, `--changed`, `--changelog PKG` and `--rollback` print a warning and exit 0 without updating. `--firmware`, `--no-firmware`, `--snapshot`, `--parallel`, `--resume`, `--download-only`, `--deep-clean` and `--refresh-mirrors` print a warning and the run goes on without them.

9.2 Combining flags

Any order, any combination. Single-letter short flags in the set `{h V n v q y w}` bundle (`up -nv` equals `up -n -v`; `up -ynv` equals `up -y -n -v`). Compound short flags (`-nl`, `-ni`, `-qq`, `-up`, `-err` and so on) and long flags are never bundled.

```
up -n -v                        # dry-run with live stream
up -y -q --security             # unattended security-only, minimal output
up --dev --parallel -v          # dev toolchains + user-space concurrency
up --snapshot -y && up --reboot   # snapshot, update, then reboot if needed
up --profile server --resume    # resume an interrupted headless run
up -n --dev --firmware          # preview a full run, no side effects
up -y -q -nl -ni                # what the scheduled timer runs
up --refresh-mirrors -y -v      # re-rank mirrors then upgrade, live
```

Flags that take a value (`--profile NAME`, `--changelog PKG`, `--metrics FILE`, `--only LIST`, `--except LIST`) take it as the next word or as `--flag=VALUE` (`--only=apt,flatpak`, `--profile=work`, `--changelog=bash`, `--metrics=FILE`, `--except=snap`). An empty value exits 2. CLI flags override config and profile, so you can tweak a profile for one run:

```
up --profile work --dev         # work profile, but force --dev this time
```

10 Features

10.1 Dry-run and verbose

`-n` prints each command instead of running it, and does not ask for elevation. Example output (apt and flatpak):

```
$ up -n
    . [dry-run] sudo apt -o APT::Status-Fd=1 --fix-broken install -y
    . [dry-run] sudo apt -o APT::Status-Fd=1 update
    . [dry-run] sudo apt -o APT::Status-Fd=1 full-upgrade -y
    . [dry-run] sudo apt -o APT::Status-Fd=1 autoremove -y
    . [dry-run] sudo apt autoclean -y
    . [dry-run] sudo apt clean
    . [dry-run] flatpak update -y --noninteractive
    . [dry-run] flatpak uninstall --unused -y --noninteractive
```

`-v` streams each manager's output live instead of collapsing it into one result line. Combine both with `up -n -v`.

10.2 Quiet levels

- Default: logo, info line, section headers with step progress, spinner, per-step timings and the summary box.
- `-q`: hides muted and informational lines; keeps the per-manager result and the summary.
- `-qq`: only the one-line summary (good for systemd logs).
- `-qqq`: nothing, only the exit code.

10.3 Security-only updates

With `--security`, managers that support it limit the upgrade to security updates:

- apt: `unattended-upgrade -v` when `unattended-upgrade` is installed; otherwise apt is skipped.
- dnf: `upgrade --security`.
- yum: `update --security`.
- zypper: `patch --category security`.
- softwareupdate (macOS): `--install --recommended` (recommended updates, not only security ones), only with `--accept-licenses`; without it the updates are listed.

On every platform every other manager is skipped with a warning, so `--security` never becomes a full upgrade; a plugin takes part when it defines `pm_<slug>_security`. On Windows, Windows Update (PSWindowsUpdate) installs only the "Security Updates" category, and only with `--accept-licenses`; without it the updates are listed.

10.4 Dev toolchains

Off by default; `--dev` turns on:

Tool	Command
rustup	rustup update
cargo	cargo install-update -a
npm	npm update -g
pnpm	pnpm -g update
pip	pip list --outdated, then user-site upgrade
gem	gem update

Never run under sudo. Missing toolchains are skipped silently.

10.5 Firmware (Linux)

--firmware runs fwupdmgr refresh && fwupdmgr update -y --no-reboot-check. On macOS firmware comes through softwareupdate; on Windows it is left to vendor tools (Dell Command Update, Lenovo Vantage). On a Raspberry Pi with an SPI EEPROM (Pi 4, 400, 5, 500, CM4, CM5) --firmware also stages the bootloader update with rpi-eeprom-update -a; it is flashed on the next reboot.

10.6 Raspberry Pi

Detected from /proc/device-tree/model.

- Power and temperature: vcgencmd get_throttled and measure_temp are read before anything is installed and shown on one line. If the board is under-voltage *right now*, eMerger warns that a brown-out while dpkg writes can corrupt the SD card and asks before continuing. With -y it warns and continues, unless RPI_ABORT_ON_UNDERVOLTAGE=1. Past under-voltage since boot, and a SoC temperature above RPI_TEMP_WARN (default 80 °C), are reported as warnings.
- Bootloader EEPROM (rpi-eeprom manager, boards with an SPI EEPROM only): every run checks for an update and reports it; --firmware applies it and the reboot advisory says it will be flashed on next boot. --no-firmware skips the check.

10.7 Live progress

A running step shows a spinner, a progress bar, elapsed time and what the manager is doing. The bar comes from parsing the manager's output as it arrives (on Windows, pslib/ui/Monitor.ps1 and pslib/managers/Progress.ps1):

Manager	What drives the bar
apt	APT::Status-Fd: files fetched during update, real percent during downloads and dpkg
pacman, yay, paru, zypper, apk	(3/24) upgrading ... counters
dnf, yum	Upgrading : pkg 3/24, dnf5 [3/24], download (3/24): lines
emerge	>> Emerging (3 of 24)
flatpak	Updating 2/5, percentages
snap, fwupd, softwareupdate, rustup	percentages
brew	==> Upgrading N outdated packages against each upgrade
pip	eMerger's own [i/N] pkg loop
eopkg	Downloading 3 / 24
gem, cargo, xbps, nix, mas	running count when no total is printed
winget (Windows)	(i/N) Found counters, download sizes and percentages

Manager	What drives the bar
choco (Windows)	per-package upgrade lines, `Progress: 45%`
scoop (Windows)	`Updating N outdated apps` against each update
PSWindowsUpdate (Windows)	accepted / downloaded / installed rows

If there is nothing to count you get the `.::::.` wave instead. `-v` shows the raw output.

10.8 Cache cleaning

The user-cache step lists the entries of `~/.cache` biggest first, with a size bar, and asks which to remove: `a` (all), `n` (none, the default) or a list such as `1 3 5-7`. Entries past `CACHE_MENU_ROWS` (12) are grouped as one "others" row. eMerger's own cache is always kept; `CACHE_EXCLUDE="mozilla,pip*"` keeps more. With `-y` everything not excluded is cleaned.

On Linux the step then reports the deleted files a running process still keeps open on the filesystems of `/` and `$HOME`, when they hold 1M or more: the five biggest holders, by command. That space comes back only when the holder closes the files or exits, so nothing is deleted; restart the program named.

10.9 Deep clean

`up --deep-clean` goes after system-level leftovers a normal run doesn't touch, asking about each one and showing how much it would free. Skip tasks with `DEEP_CLEAN_SKIP="docker,journal"`.

Key	Task
`journal`	`journalctl --vacuum-size=$JOURNAL_MAX_SIZE` (200M), only if bigger
`kernels`	dnf: keep the 2 newest kernels; zypper: `purge-kernels`
`residual`	`dpkg --purge` config files of removed packages
`snap`	remove disabled snap revisions
`coredumps`	delete `/var/lib/systemd/coredump` files
`npm`, `go`	`npm cache clean --force`, `go clean -cache`

Two more sections follow, with the same prompts and skip keys.

Key	Task
`docker`, `podman`	Containers section, per local docker context (`unix://`; remote ones are skipped): `container prune -f`, `image prune -af`, `builder prune -af` (docker only) and `volume prune` (`-af` on docker), each with its reclaimable size, a kind at 0B not offered, sizes read again after the container prune. Stopped containers (toolbox, distrobox) and volumes hold data: never pruned under `-y`. Only the Linux system docker daemon is tried through `sudo`. Docker Desktop's data (`~/.docker/desktop`, on macOS `~/Library/Containers/com.docker.docker`) is shown from 1M, since its disk image shrinks only from Docker Desktop

Respects `-n`; space freed on `/` goes into the summary.

10.10 Parallel mode

`--parallel` runs user-space managers that do not touch the root filesystem concurrently: `flatpak`, `snap`, `brew`, `mas`, and the dev toolchains. System managers (`apt`, `dnf`, `pacman`, `zypper` and so on) stay serial to avoid lock contention on `/var/lib/dpkg`, `/var/cache/pacman` and similar. Windows is always serial.

10.11 Snapshots and rollback (Linux)

`--snapshot` tries, in order:

1. `snapper`: a pre-update snapshot of the root subvolume, tagged `eMerger pre-update`.
2. `timeshift`: `timeshift --create --tags D`.
3. `btrfs subvolume snapshot`, only if neither of the above is installed. The snapshot goes under `/.snapshots/emerger/`.

If none of them works, eMerger names each failure and exits 1 before any manager runs.

`--rollback` goes back to the latest snapshot eMerger made:

- snapper: `snapper -c root rollback <num>`, effective after a reboot.
- timeshift: runs `timeshift --restore` interactively (`--yes` is not passed).
- plain btrfs: eMerger does not swap subvolumes itself; it prints the path of the latest snapshot so you can do it by hand.

10.12 Mirrors refresh (Linux)

`--refresh-mirrors` re-ranks mirrors before the upgrade:

- Arch: `reflector --latest 20 --sort rate`.
- Debian (not its derivatives): `netselect-apt`, if installed, for the running release.
- Fedora: no-op, the `fastestmirror` plugin already does this.

10.13 Reboot handling

On Unix, eMerger inspects `/var/run/reboot-required` and `needs-restarting -r` (if available). On Windows it reads these registry entries:

- `CBS\RebootPending`
- `WindowsUpdate\RebootRequired`
- `PendingFileRenameOperations`
- The summary always prints a reboot advisory.
- `--reboot` reboots the machine if needed (requires elevation).
- `--reboot-exit` exits with code 4, so a calling script can decide what to do.

10.14 Resume

After each successful manager eMerger writes a cursor to `~/.local/state/emerger/resume`. `up --resume` skips the managers that finished in the last run. A run without `--resume` starts a fresh cursor, and a dry run leaves it alone. Unix only.

10.15 Package diff and changelog

Every run records installed packages before and after. `up --changed` prints the diff (`+` added, `-` removed, `~` upgraded):

```
$ up --changed
 ~ firefox          123.0.1   ->  124.0
 ~ libc6            2.39-0    ->  2.39-1
 + flatpak-xdg-utils 1.0.6
 - old-package      2.1
```

`up --changelog PKG` shows one package's upstream changelog via `apt changelog`, `dnf changelog` (or `updateinfo`) or `brew log`. pacman has no changelog: it shows the package info (`pacman -Qi`) instead, labelled as such.

10.16 History and errors

- `--history` prints the last 10 runs as a table.
- `-err` / `--errors` tails `ERROR` lines from `~/.local/state/emerger/emerger.log`.

10.17 Doctor

`--doctor` reports each of these as OK, warning or failure:

- shell / PowerShell version;
- `sudo` cache / admin status;
- free space on `/` (`C:` on Windows) against `DISK_MIN_FREE_MB`;
- network reachability: one HTTPS request to `github.com` (Windows: one connection test to it);
- state directory writability;
- glyph mode (UTF-8, emoji font) and how to get emoji icons;
- per-manager native health: `dpkg --audit`, `pacman -Dk`, `dnf check` / `yum check`, `brew doctor`;
- pending reboot flag;
- (Windows) the current ExecutionPolicy and elevation status.

Exits non-zero if any check warns or fails. Please attach its output to bug reports.

11 Configuration

11.1 Config file (Unix)

`~/.config/emerger/config.sh` is a shell file sourced before argument parsing. Each setting flag has a variable, listed under "Every supported variable" below:

```
# ~/.config/emerger/config.sh

# Defaults
ARG_DEV=1                   # always include dev toolchains
ARG_WEATHER=1               # show weather widget via wttr.in
ARG_PARALLEL=1              # run user-space managers in parallel
ARG_KEEP_TRASH=1            # never empty the trash automatically

# Thresholds
DISK_MIN_FREE_MB=2048       # require at least 2 GiB free on /
BATTERY_MIN_PCT=25          # warn below 25%
RETRY_ATTEMPTS=3            # attempts on a transient network error
RETRY_DELAY=5               # first retry after 5s, second after 10s

# Scheduling
QUIET_HOURS="23:00-07:00"   # runs with -y exit 0 inside this window

# Caching
EMERGER_CACHE_TTL=3600      # detection cache: 1h (0 = disabled, default 86400)
```

11.2 Config file (Windows)

`%APPDATA%\emerger\config.ps1` is dot-sourced before argument parsing:

```
# %APPDATA%\emerger\config.ps1

$script:ArgsGlobal.Dev       = $true
$script:ArgsGlobal.Security  = $true
$script:ArgsGlobal.KeepTrash = $true
$script:UI_VERBOSE           = $true   # as -v

# Thresholds (same meaning as on Unix)
$script:DISK_MIN_FREE_MB = 4096
$script:RETRY_ATTEMPTS = 3
$script:MANAGER_TIMEOUT = 3600   # seconds per manager step, 0 = no limit
```

11.3 Every supported variable

One variable per flag. `config.sh` and profiles set the Unix variable to 1 or to the flag's value; `config.ps1` and Windows profiles set the field of `$script:ArgsGlobal` to `$true` or to the value. `--profile` has no variable: a profile cannot select a profile. The flags that run an action or a query instead of an update (`-h`, `-V`, `--doctor`, `--list-profiles`, `--rollback`, `--reboot`, `--changed`, `--changelog`, `--history`, `--metrics`, `-err`, `-up`, `-au`, `-rc`) are taken from the command line only: setting their variables in a file has no effect.

Flag	Unix variable	Windows field
`--doctor`	`ARG_DOCTOR`	`Doctor`
`-n`	`DRY_RUN`	`DryRun`
`-v`	`UI_VERBOSE`	`$script:UI_VERBOSE`
`-q`	`QUIET_LEVEL (0 to 3)`	`$script:QUIET_LEVEL`
`-y`	`ARG_YES`	`Yes`
`--json`	`ARG_JSON`	`Json`
`--security`	`ARG_SECURITY`	`Security`
`--accept-licenses`	`ARG_ACCEPT_LICENSES`	`AcceptLicenses`

Flag	Unix variable	Windows field
`--firmware`	`ARG_FIRMWARE`	`Firmware`
`--no-firmware`	`ARG_NO_FIRMWARE`	`NoFirmware`
`--dev`	`ARG_DEV`	`Dev`
`--parallel`	`ARG_PARALLEL`	`Parallel`
`--list-profiles`	`ARG_LIST_PROFILES`	`ListProfiles`
`--only LIST`	`ARG_ONLY`	`Only`
`--except LIST`	`ARG_EXCEPT`	`Except`
`--snapshot`	`ARG_SNAPSHOT`	`Snapshot`
`--rollback`	`ARG_ROLLBACK`	`Rollback`
`--refresh-mirrors`	`ARG_REFRESH_MIRRORS`	`RefreshMirrors`
`--resume`	`ARG_RESUME`	`Resume`
`--reboot`	`ARG_REBOOT`	`Reboot`
`--reboot-exit`	`ARG_REBOOT_EXIT`	`RebootExit`
`--download-only`	`ARG_DOWNLOAD_ONLY`	`DownloadOnly`
`-nl`	`ARG_NO_LOGO`	`NoLogo`
`-ni`	`ARG_NO_INFO`	`NoInfo`
`-w`	`ARG_WEATHER`	`Weather`
`--ascii`	`ARG_ASCII`	`Ascii`
`-nc`	`ARG_KEEP_CACHE`	`KeepCache`
`-nt`	`ARG_KEEP_TRASH`	`KeepTrash`
`--deep-clean`	`ARG_DEEP_CLEAN`	`DeepClean`
`--changed`	`ARG_CHANGED`	`Changed`
`--changelog PKG`	`ARG_CHANGELOG`	`Changelog`
`--history`	`ARG_HISTORY`	`History`
`--metrics FILE`	`ARG_METRICS`	`Metrics`

Settings. Set in `config.sh`, a profile or the environment on Unix; in `config.ps1` or a profile as `$script:NAME` on Windows, for the rows marked "all".

Variable	Default	Platforms	Meaning
`DISK_MIN_FREE_MB`	1024	all	warn and ask below this many MiB free on / or `C:`
`BATTERY_MIN_PCT`	20	all	on battery, warn and ask below this percentage
`RETRY_ATTEMPTS`	2	all	attempts on a transient network error (2 = one retry)
`RETRY_DELAY`	3	all	seconds before the first retry; the n-th waits n times as long
`MANAGER_TIMEOUT`	0	all	seconds one manager command may run (exit 124); 0 = no limit
`QUIET_HOURS`	(unset)	Unix	`"HH:MM-HH:MM"`, may wrap past midnight: a run with `-y` inside it exits 0
`CACHE_EXCLUDE`	(empty)	Unix	`~/.cache` entries never offered (names or globs)
`CACHE_MENU_ROWS`	12	Unix	rows in the cache menu before "others"

Variable	Default	Platforms	Meaning
JOURNAL_MAX_SIZE	200M	Unix	journal target for --deep-clean
DEEP_CLEAN_SKIP	(empty)	Unix	deep-clean tasks to skip (§10.9)
RPI_ABORT_ON_UNDERVOLTAGE	0	Unix	abort even with -y on Pi undervoltage
RPI_TEMP_WARN	80	Unix	Pi SoC temperature warning (°C)

Environment. Read from the environment on the platforms listed; on Unix config.sh can set them too, the XDG_* ones excepted.

Variable	Default	Platforms	Meaning
EMERGER_EMOJI	auto	all	emoji icons only with a color-emoji font; 1/0 to force
EMERGER_SPINNER	(auto)	all	braille, dots, pulse (ASCII .:), line; unset: braille on a UTF-8 terminal, pulse otherwise
EMERGER_PROMPT_TIMEOUT	60	all	seconds a y/N prompt waits before taking the default (0 = forever)
EMERGER_CACHE_TTL	86400	Unix	detection-cache TTL in seconds; 0 disables
NO_COLOR	(unset)	all	any value turns colors off
COLORFGBG	(unset)	all	fg;bg from the terminal: bg 7 or more picks the light-background palette
XDG_CONFIG_HOME	(unset)	Unix	parent of the config directory; ~/.config when unset
XDG_CACHE_HOME	(unset)	Unix	parent of the cache directory; ~/.cache when unset
XDG_STATE_HOME	(unset)	Unix	parent of the state directory; ~/.local/state when unset

Numeric variables take whole numbers only; any other value warns and the default is used. On Windows a value above 999999999 is capped there. The safety switches DRY_RUN and RPI_ABORT_ON_UNDERVOLTAGE exit 2 on any other value instead. Without timeout, MANAGER_TIMEOUT warns once and is not applied. config.sh, user profiles, plugins and hooks are loaded only when you own them and no other user can write them (a file writable by your own private group is accepted). Running as root with a HOME owned by another user, as sudo -E up does, is refused.

CLI flags are applied after the config and the profile and win. Every flag sets its variable, except --except, which adds to it: ARG_EXCEPT=snap in config.sh plus up --except flatpak skips both. --no-firmware wins over ARG_FIRMWARE=1.

11.4 Profiles

A profile is a config snippet loaded with --profile NAME (up --list-profiles lists them). Shipped in share/profiles/:

Profile	What it sets
`work`	security only, unattended, cache and trash left alone; Unix also: no firmware, skips yay and paru
`home`	everything: dev toolchains; Unix also: firmware, parallel mode and the weather line
`server`	headless: quiet (`-qq`), security only, no prompts, logo or info line, cache and trash left alone; Unix also: no firmware
`safe`	least change: security only, unattended, cache and trash left alone; Unix also: no firmware, skips flatpak and snap

Your own profiles go in `profiles.d` under the config directory (`~/.config/emerger/` or `%APPDATA%\emerger\`) and take precedence over shipped ones with the same name. A `--except` on the command line adds to the profile's `ARG_EXCEPT` instead of replacing it, and the shipped profiles add to an `ARG_EXCEPT` set in `config.sh`.

The shipped `work` profile:

```
# shellcheck shell=bash
# description: security only, unattended, no firmware, skips yay and paru
ARG_SECURITY=1
ARG_YES=1
ARG_NO_FIRMWARE=1
ARG_EXCEPT="${ARG_EXCEPT:+$ARG_EXCEPT,}yay,paru"
ARG_KEEP_CACHE=1
ARG_KEEP_TRASH=1
```

A custom one, used with `up --profile train`:

```
# ~/.config/emerger/profiles.d/train.sh
# description: prefetch only, quiet output
ARG_DOWNLOAD_ONLY=1
ARG_YES=1
QUIET_LEVEL=2
ARG_KEEP_TRASH=1
ARG_KEEP_CACHE=1
```

12 Hooks

Scripts in `hooks/pre.d/` (before updates) and `hooks/post.d/` (after), run in alphabetical order. A failing hook prints a warning and the run goes on. Unix hooks are `*.sh` files, run with `bash` (no executable bit needed); Windows hooks are `*.ps1` files, run in a child scope.

12.1 Example: back up dotfiles before every run

```
# ~/.config/emerger/hooks/pre.d/10-backup-dotfiles.sh
#!/usr/bin/env bash
set -e
rsync -a --delete ~/.config/ ~/backups/dotfiles/
```

12.2 Example: Slack notification after every run

```
# ~/.config/emerger/hooks/post.d/99-slack-notify.sh
#!/usr/bin/env bash
state=~/.local/state/emerger
```

```
tail -1 "$state/history.jsonl" | \
  curl -sS -X POST -H 'Content-Type: application/json' \
    --data-binary @- "$SLACK_WEBHOOK_URL" >/dev/null
```

12.3 Example: copy the last run log to the clipboard (Windows)

```
# %APPDATA%\emerger\hooks\post.d\10-log-to-clipboard.ps1
$log = Join-Path $env:LOCALAPPDATA 'emerger\state\emerger.log'
Get-Content $log -Tail 40 | Set-Clipboard
```

Hooks run with the privileges of whoever ran `up`, never as root. When a detected manager needs sudo, eMerger caches the credentials before the `pre.d` hooks, so a hook can call `sudo -n`; in a run that needs no sudo, `sudo -n` fails. A dedicated `sudoers` entry works in both cases. A dry run lists the hooks and runs none.

12.4 Ignore list (Linux)

`~/.config/emerger/ignore.list` accepts one package per line, with `#` comments. Per manager:

Manager	Behavior
`pacman`	Honoured natively via `--ignore=` on the command line.
`apt`	Advisory only; also run `apt-mark hold PKG`.
`dnf`	Advisory only; also run `dnf versionlock add PKG`.
`zypper`	Advisory only; also run `zypper al PKG`.

12.5 Quiet hours

`QUIET_HOURS="HH:MM-HH:MM"` in `config.sh`. A run that starts inside the window with `-y` set (as timer runs are) exits immediately; runs without `-y` are not affected. Windows across midnight (`"23:00-07:00"`) work. A value in another format is ignored with a warning.

13 Manager plugins

To add a package manager without touching the repo, drop a bash script named `<slug>.sh` in `~/.config/emerger/managers.d/`.

13.1 Minimal plugin

```
# ~/.config/emerger/managers.d/mytool.sh
PM_PLUGIN_SLUG=mytool

pm_mytool_detect()     { command -v mytool >/dev/null 2>&1; }
pm_mytool_needs_sudo() { return 1; }     # optional, default: no sudo
pm_mytool_parallel()   { return 0; }     # optional, default: serial
pm_mytool_dev()        { return 1; }     # optional, default: not --dev gated
pm_mytool_icon()       { printf '\xf0\x9f\x94\x8c'; }   # optional
pm_mytool_security()   { return 1; }     # optional, 0: run honours ARG_SECURITY

pm_mytool_run() {
    run_cmd "mytool refresh"  mytool refresh  || return 1
    run_cmd "mytool upgrade"  mytool upgrade -y || return 1
    return 0
}
```

13.2 Plugin API

The file sets `PM_PLUGIN_SLUG` and defines functions named after it. A file without the slug, `pm_<slug>_detect` or `pm_<slug>_run` is skipped with a warning.

Name	Required	Contract
`PM_PLUGIN_SLUG`	yes	manager name, as shown and as matched by `--only`
`pm_<slug>_detect`	yes	return 0 if the manager is installed
`pm_<slug>_run`	yes	refresh, upgrade, clean; return 0 on success
`pm_<slug>_needs_sudo`	no	return 0 if it needs root (default: no)
`pm_<slug>_parallel`	no	return 0 if safe under `--parallel` (default: serial)
`pm_<slug>_dev`	no	return 0 to run only with `--dev`
`pm_<slug>_security`	no	return 0 if `pm_<slug>_run` honours `ARG_SECURITY`; without it `--security` skips the plugin
`pm_<slug>_icon`	no	print a short glyph
`run_cmd LABEL CMD...`	helper	run one command with the features below

13.3 Behavior

Plugins behave like built-in managers: `--only`, `--except`, `--parallel`, `--dev`, the detection cache, hooks and the summary all apply to them. Commands run through `run_cmd` also get:

- `--dry-run` support;
- retry on transient failures (`RETRY_ATTEMPTS`, `RETRY_DELAY`);
- logging to `~/.local/state/emerger/emerger.log`;
- live-log streaming under `-v`.

Full template: `share/plugins/example.sh`.

13.4 Detection cache

Detection results are cached per manager slug in `~/.cache/emerger/detected` for 1 day. Set `EMERGER_CACHE_TTL=<seconds>` in `config.sh` to change this (0 disables the cache). Run `up -rc` after installing or removing a package manager.

Windows loads no plugins.

14 Integration

14.1 JSON output

`up --json` prints one line of JSON on stdout, with no logo, info line or summary box, so you can pipe it into `jq`:

```
{"ts":"2026-04-14T07:24:31Z","duration":42,"freed_kb":15360,
 "errors":0,"reboot":0,
 "managers":[{"name":"apt","result":"ok","secs":30},{"name":"flatpak","result":"ok","
    secs":12}]}
```

Hook output, plugin output and, with `-v`, manager output go to stderr, so stdout holds the JSON line alone.

Every run except a dry run also appends this line to `~/.local/state/emerger/history.jsonl` (`%LOCALAPPDATA%\emerger\state\history.jsonl` on Windows). Both platforms write the same fields.

Field	Meaning
`ts`	ISO 8601 UTC timestamp of the run end
`duration`	total wall-clock seconds
`freed_kb`	cache and trash bytes freed, in KiB
`errors`	number of managers that returned non-zero
`reboot`	1 if a reboot is required, else 0
`managers`	array of `{name, result, secs}` entries, `secs` being that manager's wall-clock seconds

14.2 Prometheus metrics

`up --metrics FILE` writes the latest `history.jsonl` entry in Prometheus textfile-collector format:

- `emerger_last_run_timestamp_seconds`
- `emerger_last_run_duration_seconds`
- `emerger_last_run_freed_bytes`
- `emerger_last_run_errors`
- `emerger_reboot_required`
- `emerger_manager_ok{manager="..."}` (one per manager)

It doesn't start a run; call it from a `post.d` hook or after `up` in your timer. A field missing from the entry reads 0. On Windows a last line that is not valid JSON exits 1.

```
# ~/.config/emerger/hooks/post.d/90-prom.sh
up --metrics /var/lib/node_exporter/textfile_collector/emerger.prom
```

14.3 Reboot exit code

A successful run exits 0 even with a reboot pending (the summary says `Reboot recommended`). With `--reboot-exit` it exits 4 instead:

```
up -y --reboot-exit
case $? in
    0) ;;                                   # done, no reboot needed
    3) notify-send "eMerger: some managers failed" ;;
    4) systemctl reboot ;;                  # clean, reboot required
esac
```

14.4 Download-only

`--download-only` refreshes indexes and downloads pending upgrades without installing them:

Manager	Command
pacman	pacman -Syuw
apt	apt full-upgrade -y --download-only (dist-upgrade with apt-get)
dnf	dnf upgrade -y --downloadonly
zypper	zypper update --download-only
softwareupdate	softwareupdate --download --all (--recommended with --security)

The apt repair step (`--fix-broken install`) also runs with `--download-only`. Every other manager, plugins included, is skipped with a message, so nothing installs.

14.5 Manager filtering

```
up --only apt,flatpak        # only these managers
up --except snap,fwupd       # everything else
```

Filters apply after detection and after `--dev` / `--firmware` gating, so:

- `--only X` runs nothing if `X` is not detected.
- `--except` beats `--only` when both name the same manager.
- `--except` adds to a list from `config.sh` or a profile instead of replacing it (`up --profile safe --except npm` skips flatpak, snap and npm).

15 Auto-update (unattended)

```
up -au               # Linux / macOS
```

```
up -au               # Windows
```

- Linux: a systemd user timer at `~/.config/systemd/user/emerger.{service,timer}`, or cron when `systemctl --user` is not available.
- macOS: cron, via `crontab -l` (set up `launchd` yourself if you prefer it).
- Windows: `Register-ScheduledTask -TaskName eMerger`. When an admin manager (choco, wsl, Windows Update) is present, the run raises a UAC prompt at the scheduled time and keeps the run lock until it is answered.

Weekly, Sunday 10:00 local time (up to one hour later at random, except under cron), always with `-y -q -nl -ni`. On Linux and macOS, use `QUIET_HOURS` to keep runs out of a time window. On Linux, systemd user timers don't fire without an open login session; run `loginctl enable-linger $USER` once (as root) to fix that.

15.1 Managing the schedule

```
# Linux
systemctl --user status emerger.timer
systemctl --user stop emerger.timer
systemctl --user disable emerger.timer
```

```
# Windows
Get-ScheduledTask eMerger
Unregister-ScheduledTask eMerger
```

16 Cookbook

16.1 Interactive run

```
up -v
```

Add `--firmware` to include firmware.

16.2 Unattended server

Install the schedule once:

```
cat > ~/.config/emerger/config.sh <<'EOF'
ARG_SECURITY=1
ARG_YES=1
ARG_KEEP_CACHE=1
ARG_KEEP_TRASH=1
QUIET_HOURS="08:00-18:00"
EOF
up -au
```

For monitoring, add a `post.d` hook with `up --metrics` (section 14).

16.3 Dev toolchains and firmware

```
up --dev --firmware --parallel -v     # or: up --profile home
```

16.4 Least change

Security patches only, unattended, no firmware:

```
up --profile safe
```

16.5 Download now, install later

```
up --download-only -y     # on a good connection
up -y                     # later, installs from the package cache
```

16.6 CI runner and container image build

```
up --only apt -y -qq --reboot-exit
rc=$?
if (( rc == 4 )); then echo "reboot required"; exit 0; fi
exit $rc
```

16.7 Snapshot and rollback

```
set -e
up --snapshot -y || { up --rollback; exit 1; }
up -y --reboot-exit
if [[ $? -eq 4 ]]; then systemctl reboot; fi
```

16.8 One package

```
up -n | grep -i firefox          # what would happen to it
up --changelog firefox           # what changed upstream
```

To pin it, see the ignore list (section 12.4).

17 Security model

- Runs when you call it, or from the timer installed with `-au`. No open ports. During a run it keeps two helper processes, both gone when the run ends: the progress monitor and, when a manager needs `sudo`, a refresher that renews the `sudo` ticket every 60 seconds.
- Its own state lives in `~/.config/emerger/`, `~/.cache/emerger/` and `~/.local/state/emerger/` (the lock file `emerger.lock` included). State and cache directories are private (mode 0700).
- It also writes, only when asked: the `up` alias and completion files (`install.sh`), a systemd user timer or crontab line (`-au`), the mirror list (`--refresh-mirrors`), a snapshot (`--snapshot`), the `--metrics` file, and deletions inside `~/.cache` and the Trash when cleaning. `--deep-clean` also deletes journal entries, old kernels, leftover configs, snap revisions, crash dumps (`/var/lib/systemd/coredump`), unused containers, images, build cache and volumes, after asking for each.
- On Windows, UAC elevation happens only for the managers that need admin. They run in an elevated child that loads no config, profile or hook. That child runs the code of the checkout: clone it into a folder only administrators can write to if other programs running as you must not reach admin rights. The child starts with `PATH` and `PSModulePath` reduced to system folders, so a tool or module in a folder you can write is not run elevated. `ComSpec` and `PATHEXT` are reset, `ChocolateyInstall` comes from the machine scope, and the `DOTNET_*`, `COR_*`, `CORECLR_*`, `COMPlus_*` and `PSExecutionPolicyPreference` variables are removed before the managers run. Variables that act when a process starts (a profiler or startup hook set in your user environment) have already acted on the elevated PowerShell by then: keep your user environment free of them. A `choco` or `wsl` outside Windows, System32, Program Files, or a `ProgramData\chocolatey` that standard users can write, is refused. `choco upgrade all -y` accepts package licences and runs only with `--accept-licenses`; without it `choco outdated` lists the upgrades. Every child PowerShell (elevated child, step runner, scheduled task) starts with `-ExecutionPolicy Bypass`, so the `RemoteSigned` policy does not apply to `up`'s own scripts.
- Linux and macOS: config files, profiles, plugins and hooks are refused when another user could change them. The file and its directory must belong to you and not be writable by group or others (your own private group excepted).
- The checkout runs as you, and as root under `sudo`. Keep it writable by you alone: `up` refuses to start when a file under `src/` is writable by group or others (your private group excepted).
- Hooks and plugins run as you and can do anything you can. Only install ones you trust.
- With `-v` the managers write to the terminal directly; otherwise their output is stripped of terminal control sequences.
- eMerger's own network traffic: the optional weather line (`wttr.in`), `git pull` for `--self-update`, `fwupdmgr refresh` with `--firmware`, and one connectivity check to `github.com` during `--doctor`, and with `--refresh-mirrors` the mirror lists that `reflector` and `netselect-apt` fetch. Everything else comes from the package managers.

- No telemetry. `history.jsonl` stays local unless one of your hooks sends it somewhere.

18 Files and paths

Linux and macOS

Path	Purpose
`~/.config/emerger/config.sh`	User defaults
`~/.config/emerger/profiles.d/`	User profiles
`~/.config/emerger/hooks/{pre,post}.d/`	Hook scripts
`~/.config/emerger/ignore.list`	Ignore list (pacman native)
`~/.config/emerger/managers.d/*.sh`	User manager plugins
`share/profiles/`	Shipped profiles, in the checkout
`~/.cache/emerger/detected`	Detection cache (directory mode 0700)
`~/.local/state/emerger/emerger.log`	Log (rotated at 2000 lines, mode 0600, directory 0700)
`~/.local/state/emerger/history.jsonl`	One JSON record per run
`~/.local/state/emerger/pkgs.before`	Package list snapshot (pre)
`~/.local/state/emerger/pkgs.after`	Package list snapshot (post)
`~/.local/state/emerger/pkgs.diff`	Last-run package diff
`~/.local/state/emerger/resume`	Resume cursor
`~/.local/state/emerger/emerger.lock`	Global lock (flock; `emerger.lock.d` without it)
`~/.local/state/emerger/parallel-*.log`	Output of each manager of a `--parallel` run

Windows

Path	Purpose
`%APPDATA%\emerger\config.ps1`	User defaults
`%APPDATA%\emerger\profiles.d\`	User profiles
`%APPDATA%\emerger\hooks\pre.d\`	Hook scripts run before the managers
`%APPDATA%\emerger\hooks\post.d\`	Hook scripts run after the managers
`share\profiles\`	Shipped profiles, in the checkout
`%LOCALAPPDATA%\emerger\state\emerger.log`	Log
`%LOCALAPPDATA%\emerger\state\history.jsonl`	Run history
`%LOCALAPPDATA%\emerger\state\run.lock`	Per-user run lock
`%LOCALAPPDATA%\emerger\state\elevated.lock`	Lock the elevated child holds while its managers run

19 Supported package managers

- **Linux, system (need sudo):** `pacman`, `apt`, `apt-get`, `dnf`, `yum`, `zypper`, `xbps`, `apk`, `eopkg`, `emerge`, `nixos-rebuild`, `fwupdmgr`, `snap`, `rpi-eeprom-update` (Raspberry Pi bootloader).
- **Linux, AUR (no sudo):** `yay`, `paru`.
- **Linux, user-space:** `flatpak`, `nix-env`.
- **macOS:** `softwareupdate` (native), `brew`, `brew --cask`, `mas`.

- **Windows:** `winget`, `scoop`, `choco`, `PSWindowsUpdate`, `wsl --update`.
- **Dev toolchains** (all platforms, opt-in with `--dev`): `rustup`, `cargo install-update`, `npm`, `pnpm`, `pip`, `gem`.

20 Exit codes

Code	Meaning
0	Success
1	Runtime failure (sudo, lock, disk, interrupted) or aborted (a declined safety prompt, no snapshot taken with `--snapshot`)
2	Argument parsing error
3	One or more package managers returned non-zero
4	Reboot required (only with `--reboot-exit`)

```
up -y -q || case $? in
    3) notify-send "eMerger: some managers failed" ;;
    *) logger -t eMerger "fatal $?" ;;
esac
```

```
up -y -q
if ($LASTEXITCODE -eq 3) { Write-Warning "Some managers failed" }
```

21 Troubleshooting

"Another eMerger run is in progress". Another `up` holds the lock, and the message names its PID. Wait for it, or stop that process (`kill PID`). Deleting the lock file does not release a `flock`: it lets a second run start beside the first. A process started by a hook or a manager never holds the lock. On Windows the message names the lock file, not a PID: wait for the other `up` window (or the scheduled task) to finish. An elevated window left running by an interrupted `up` also holds off the next run of the same account until it ends; the elevated child creates its lock in the state directory of the account it runs under and does not start without it.

A manager fails with exit code 3. `up --errors`, then `up --doctor`. Usually it's the manager's own state (e.g. an interrupted `dpkg`); run its repair command (`sudo dpkg --configure -a`, `sudo pacman -Dk`) and retry.

Nothing happens on scheduled runs. Check `systemctl --user status emerger.timer` or Task Scheduler, and the `enable-linger` note in section 15. To see the output of a scheduled run, add `-v` in the unit once.

Emoji renders as boxes or question marks. Emoji icons are used only when fontconfig reports a color-emoji font. Install one (`sudo apt install fonts-noto-color-emoji` on Debian/Raspberry Pi OS), force the mode with `EMERGER_EMOJI=1` or `EMERGER_EMOJI=0`, or use `--ascii`. `up --doctor` shows the active mode.

The run seems stuck at "User cache" or "Trash". It's waiting at a y/N prompt (read from `/dev/tty`, defaults to "no" after `EMERGER_PROMPT_TIMEOUT` seconds). `-y` skips it.

"cannot read the Trash (macOS privacy settings)". macOS hides `~/.Trash` from a terminal without Full Disk Access. Grant it to your terminal in System Settings, Privacy & Security, Full Disk Access.

Warnings appear even with `--json`. They go to stderr, so stdout is only the JSON. Add `2>/dev/null`.

`up --self-update` aborts with "non fast-forward". You have local commits on top of `main`. Rebase or reset yourself.

`notify-send` (Linux) never shows a notification. No `DISPLAY` / `WAYLAND_DISPLAY` (usual under cron). Use the systemd user timer, which inherits the session.

(Windows) "up : The term 'up' is not recognized". The profile wasn't loaded. Run `. $PROFILE` or open a new window; if that fails, run `.\install.ps1` again.

(Windows) "cannot be loaded because running scripts is disabled". `ExecutionPolicy` is `Restricted`. Run:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

(Windows) choco, wsl or Windows Update fail with "elevation failed". The UAC prompt was cancelled. Those managers count as failed (exit 3), the rest of the run goes on. Run `up` again and accept the prompt.

Pacman warns about ignored packages. That's pacman's own message for `--ignore=`.

22 Frequently asked questions

Can I use it on production servers? Yes: `up --profile server` (unattended, security-only). Add `--reboot-exit` and let your own tooling decide when to reboot.

Does `up` install packages I don't have? No. It only upgrades what's installed, with managers that are installed. New packages show up only as new dependencies of an upgrade.

Does `up` reboot my machine? Only with `--reboot`.

Can I use it without `sudo`? Yes, limit it to user-level managers:

```
up --only flatpak,brew,npm,pip,cargo
```

Why does the terminal flicker with `--parallel -v`? Every user-space manager streams output at once. Drop `-v`, or narrow the set with `--only`.

Can I use it in a Docker image build? Yes: `up --only apt -y -qq` (or your manager). Skip `--firmware` and `--snapshot`, and ignore the reboot advisory.

23 Development

23.1 Repository layout

```
eMerger/
  src/
    emerger.sh              # Unix entry (Linux + macOS)
    emerger.ps1             # Windows entry
    lib/                    # bash libs
    pslib/                  # PowerShell libs
    logo/
  share/profiles/           # shipped profiles (*.sh + *.ps1)
  share/plugins/            # example plugins
  completions/              # bash/zsh/fish completions
  tests/                    # bats tests
  man/up.1                  # man page
  doc/documentation.tex     # this manual
  reference/                # third-party audit
  install.sh  install.ps1
  uninstall.sh  uninstall.ps1
  VERSION
  .github/workflows/ci.yml
```

23.2 Bash library modules (`src/lib/`)

File	Role
`load.sh`	Module loader: every module in load order, or the ones named
`args.sh`	Argument parser
`knobs.sh`	Numeric knob and safety switch validation
`sys.sh`	OS, shell, root, free space and process probes
`log.sh`	Structured logging (rotated), `--errors`
`trust.sh`	Owner and mode checks on user files
`hooks.sh`	User hook runner
`ignore.sh`	Ignore list loader
`profiles.sh`	Profile loader
`doctor.sh`	`--doctor`
`summary.sh`	Final box and `--json` line
`quiet_hours.sh`	`QUIET_HOURS` gate
`selfupdate.sh`	`git pull` self-update
`autoupdate.sh`	Weekly systemd timer or cron line
`help.txt`	Help text
`ui/term.sh`	Colour, UTF-8 and emoji detection, glyphs, width, cursor
`ui/text.sh`	Escape stripping, visible width, percentage bar, highlighting
`ui/print.sh`	Section headers, status lines, logo, box
`ui/monitor.sh`	Live monitor: spinner, wave, bar
`ui/prompt.sh`	Free answer and y/N prompts
`run/command.sh`	`run_cmd`: dry run, timeout, retry, captured output
`run/select.sh`	Which detected managers a run updates
`run/parallel.sh`	`--parallel` group
`run/sudo.sh`	sudo keep-alive
`managers/registry.sh`	Manager list and capabilities
`managers/detect.sh`	Detection and its cache
`managers/icon.sh`	Section icons
`managers/recipes.sh`	`pkg_run` and one recipe per manager

File	Role
managers/rpi_eeprom.sh	Raspberry Pi bootloader recipe
managers/plugins.sh	User plugin loader
managers/progress.sh	Output summary, live progress parsers (progress_feed)
managers/changelog.sh	--changelog PKG
system/preflight.sh	Disk-space and battery prechecks
system/rpi.sh	Raspberry Pi model, power and thermal pre-flight
system/reboot.sh	Reboot-required detection
system/snapshot.sh	snapper/timeshift/btrfs
system/mirrors.sh	Mirror rank/refresh
system/notify.sh	Desktop notifications
state/history.sh	history.jsonl and --history
state/estimate.sh	Step ETA from history
state/metrics.sh	Prometheus textfile export
state/diff.sh	Package snapshots + diff
state/resume.sh	Resume cursor
state/lock.sh	Global flock, mkdir fallback
clean/size.sh	Size of a path, short size format
clean/cache.sh	Selective ~/.cache menu
clean/open.sh	Report of deleted files still open
clean/trash.sh	Trash
clean/deep.sh	--deep-clean tasks
clean/containers.sh	--deep-clean Containers section

23.3 PowerShell library modules (src/pslib/)

File	Role
Args.ps1	Arg parser (shift + regex), Unix-only flag warnings
Sys.ps1	OS, architecture, admin, numeric knobs, weather
Log.ps1	Structured logging, --errors
Run.ps1	Run-Cmd: child process, live output, timeout, retry
Elevated.ps1	Elevated child for the admin managers
Summary.ps1	Final box and --json line
Hooks.ps1	hooks\{pre,post}.d*.ps1
Profiles.ps1	Profile loader
Doctor.ps1	--doctor
SelfUpdate.ps1	git pull self-update
AutoUpdate.ps1	Task Scheduler registration
Help.txt	Help text
ui/Term.ps1	Console setup, colors, glyphs, width
ui/Text.ps1	Escape stripping, visible width, elapsed time
ui/Print.ps1	Section headers, status lines, logo, box
ui/Monitor.ps1	Live monitor: spinner, wave, bar
ui/Prompt.ps1	y/N prompt
managers/Packages.ps1	Manager registry, detection, selection, Pkg-Run recipes
managers/Progress.ps1	Live progress parsers for the Windows managers (Progress-Feed)
system/Preflight.ps1	Battery and free disk checks
system/Reboot.ps1	Pending reboot detection and advisory
system/Notify.ps1	BurntToast (optional)
state/Lock.ps1	Per-user run lock, elevated child lock
state/History.ps1	history.jsonl and --history
state/Estimate.ps1	Step ETA from history
state/Metrics.ps1	Prometheus textfile export
clean/Size.ps1	Size formatting
clean/Temp.ps1	%TEMP% folders
clean/RecycleBin.ps1	Recycle Bin

23.4 Running tests

With bats-core and ShellCheck on PATH:

```
bats tests/
shopt -s globstar
shellcheck -S error src/emerger.sh src/lib/**/*.sh install.sh uninstall.sh
```

Each tests/<feature>.bats covers one feature: cli.bats the flags, lock.bats the run lock, clean.bats the cache, trash and --deep-clean, and so on. tests/test_helper.bash holds the shared fixture (temp HOME, stubs on PATH, lib to source modules). The tests/ps_*.bats files cover the PowerShell half and skip without pwsh. All of them run on push via .github/workflows/ci.yml.

23.5 Adding a package manager

Unix: edit src/lib/managers/.

1. Add the name to PKG_MANAGERS (registry.sh).
2. Add a branch in _pkg_detect_raw (detect.sh).

3. Add a `_pkg_run_<name>` recipe with `run_cmd` calls and its case in `pkg_run` (`recipes.sh`).
4. If the manager does not need sudo, exclude it from `pkg_needs_sudo` (`registry.sh`).
5. Optional: an emoji (and plain-symbol fallback) in `pkg_icon` (`icon.sh`), a stat parser in `progress_summarize`, a live-progress case in `progress_feed` (`progress.sh`) and a completion entry.

Windows: edit `src/pslib/managers/Packages.ps1`.

1. Add the name to `$script:PKG_MANAGERS` (or `PKG_DEV`).
2. Add a `Pkg-Detect` case.
3. Add a `Pkg-Run` case with `Run-Cmd` calls.
4. If it needs admin, add it to `Pkg-Need-Admin`.

Both dispatchers are table-driven, so a new manager is usually a short patch plus a test under `tests/`.

23.6 Contributing

Issues and pull requests go to GitHub, against `dev`. Before opening a PR:

- Run the test suite in `tests/` on the platform you touched.
- Attach `up --doctor` output and the relevant chunk of the log when reporting bugs.
- If a feature applies to both Bash and PowerShell, implement it on both or say in the PR that the other side is missing.